NPU-Native Neural Networks on AMD XDNA 2

Built with IRON/MLIR-AIE — target device: AMD Ryzen AI 9 HX 370 (Strix Point)

Hardware-software co-design for close-to-metal neural network inference

Abstract

This document is a compact tutorial and research note about co-designing a neural network for the AMD XDNA 2 NPU. The working example is deliberately simple: a 32-layer residual MLP for MNIST, with one learnable weight matrix per compute tile and activations flowing through the full 32-tile array in a snake. The point is not that MNIST is hard, nor that this particular design reaches the hardware's peak throughput — it does not. The point is that it is small enough to understand end to end, yet rich enough to expose the real constraints that matter on this hardware: local SRAM, MMUL-friendly data layouts, explicit FIFOs, and the cost of host–device orchestration.

1 Introduction

Modern NPUs are most interesting when they stop looking like "small GPUs" and start looking like what they are: spatial dataflow machines. On AMD XDNA 2, the programmer sees a 2D array of compute tiles, each with local SRAM, a VLIW + SIMD core, a matrix-multiply unit, and explicit data-movement primitives. That makes this hardware unusually transparent. It also means that a model that looks natural in PyTorch may look awkward once you try to map it directly onto the array.

This repository takes the hardware seriously. Instead of beginning with a large model and then asking how to squeeze it onto the device, we begin with the tile array and ask: what is the cleanest learnable network that maps onto it almost literally? The answer in the current main branch is a 32-layer residual MLP for MNIST:

$$h_{\ell+1} = \mathrm{ReLU}(h_\ell W_\ell) + h_\ell, \qquad \ell = 0, \ldots, 31.$$

Each of the 32 compute tiles stores exactly one weight matrix. The host CPU runs the input embedding and the final classification head. The NPU runs the 32 hidden layers in one call.

The result is a didactic but fully working example:

- the CPU model reaches about 97.1% MNIST test accuracy,
- the NPU implementation reaches 97.2%,
- the NPU stage runs at about 24,114 images/s for batches of 8,
- a device-side benchmark reports about 55 GFLOPS for the 32-tile snake pipeline.

These numbers are intentionally modest relative to the NPU's peak. The current paper is therefore not a claim that this is the fastest possible use of XDNA 2. It is a clear case study in co-design: how the shape of a network changes once local memory, tiling, and routing become first-class design constraints. Older, more experimental iterations of the project are preserved in `logbook.md` and on the `experimental` branch; this whitepaper focuses only on the current `resmlp/` implementation.

Mental model	CPU / GPU intuition	XDNA 2 intuition
Compute	A few large kernels over shared memory	Many small tile-local kernels connected by explicit streams
State	Parameters and activations live in DRAM / cache / HBM	Parameters should fit in tile SRAM whenever possible
Data movement	Often implicit (cache hierarchy, kernel launch runtime)	Explicit (DMA, ObjectFIFOs, placement, routing)
Architecture choice	Mostly driven by model quality	Driven by both model quality and physical layout

Table 1: The whitepaper assumes the reader is comfortable with neural networks, but new to spatial dataflow hardware. The central shift is from "launch bigger kernels" to "shape the network so each tile has a clean local job."

Property	Value
Compute array	32 compute tiles = 8 columns × 4 compute rows
Per compute tile	~64 KB local data memory + tile-local program memory
MemTile row	8 memory tiles, 512 KB each
Compute primitives	VLIW + SIMD core and bf16/BFP16-friendly MMUL instructions
Peak headline numbers	~25 TFLOPS (bf16), 50 TOPS (int8) [4, 5]
Software stack	IRON / MLIR-AIE / XRT [1, 2, 3]

Table 2: Only a few hardware facts matter for this paper. The important one is not the marketing TOPS number, but the very small per-tile SRAM budget.

2 The hardware model we program against

2.1 Why spatial dataflow changes the design problem

A CPU or GPU programmer usually thinks in terms of a few large kernels operating against a shared memory system. XDNA 2 rewards a different mental model: keep small state local to each tile, move only what must move, and make the routing pattern of the computation explicit.

For this reason, the first design question in this project is not "what is the best architecture for MNIST?" but rather "what architecture maps cleanly onto a 32-tile array with ~64 KB of SRAM per tile?" That shift is what makes the project interesting.

2.2 XDNA 2 at a glance

Figure 1 shows the physical layout we target. The Ryzen AI 9 HX 370 exposes 32 compute tiles arranged as 8 columns by 4 compute rows. Below them sit 8 MemTiles, and below those 8 shim tiles that connect the array to host-visible memory and runtime control. The main branch of this repository uses all 32 compute tiles.

2.3 One compute tile, from a programmer's point of view

Figure 2 is a logical view of one compute tile. It is not a physical floorplan. It is the subset of the architecture that matters when writing code with IRON and MLIR-AIE.

Three remarks are worth keeping in mind.

1. The most valuable resource is not peak arithmetic throughput; it is the ability to keep parameters local in SRAM.
2. The MMUL datapath likes blocked 8×8 layouts, which is why this project insists on dimensions divisible by 8 and uses explicit tiling on the host side.
3. Data movement is explicit. Activations do not "just appear" on the next tile. They move because FIFOs and DMA descriptors were created for them.

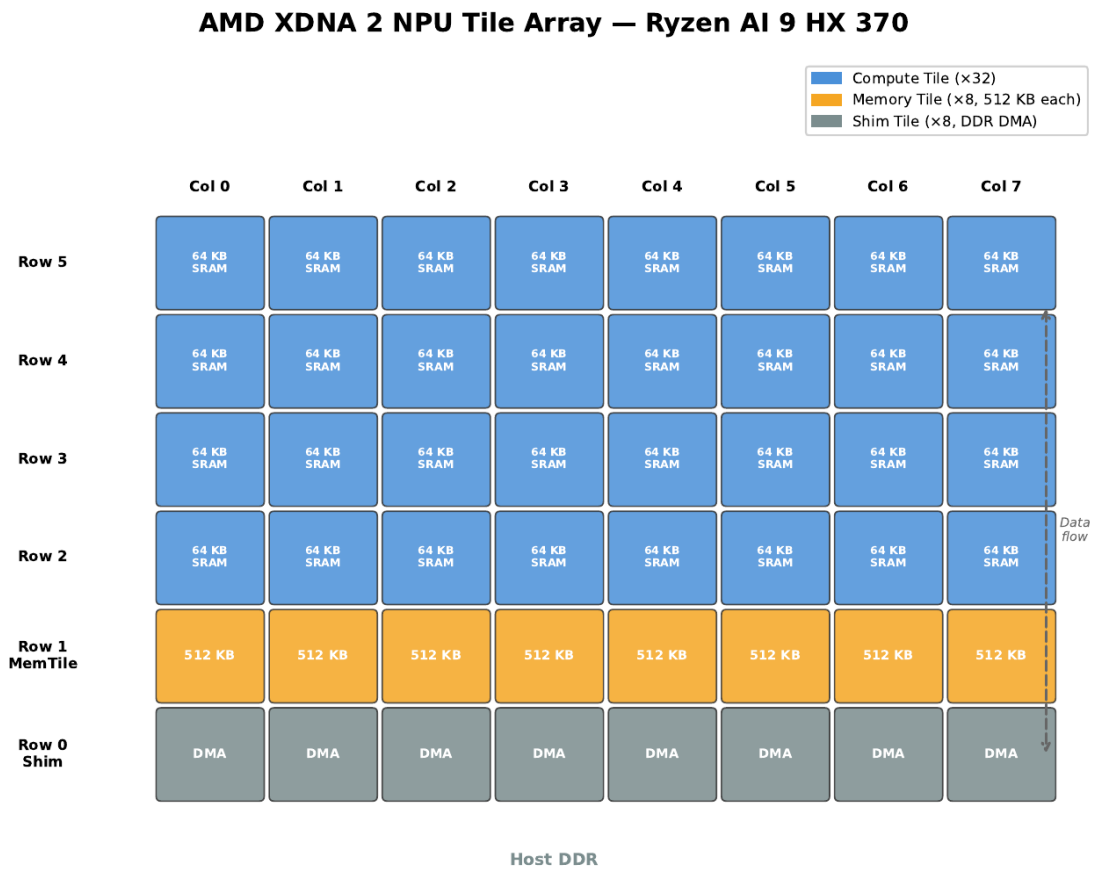

Figure 1: Physical layout of the AMD XDNA 2 NPU used here. The project uses all 32 compute tiles (8 columns × 4 rows). MemTiles provide additional on-chip memory and routing support; shim tiles connect the array to host memory and runtime control.

2.4 The constraints that actually shape the design

The current `resmlp/` implementation is driven by a short list of very practical constraints, shown in Table 3.

3 A network that matches the array

3.1 Model overview

The current model is intentionally plain. On the CPU, we flatten a 28×28 MNIST image and project it to a 160-dimensional hidden state. On the NPU, we apply 32 copies of the same layer shape, each with its own weights. On the CPU again, we project the final hidden state to ten logits.

This split is not arbitrary. The embedding and head are small and convenient to keep on the CPU. The hidden stack is the repetitive part: it is where a tile array is easiest to use well, because the same buffer shape recurs 32 times.

The PyTorch model in `resmlp/model.py` is therefore almost identical to the hardware view:

```
class ResidualLinear(nn.Module):
    def forward(self, x):
        return torch.relu(x @ self.weight) + x

class ResMLP(nn.Module):
```

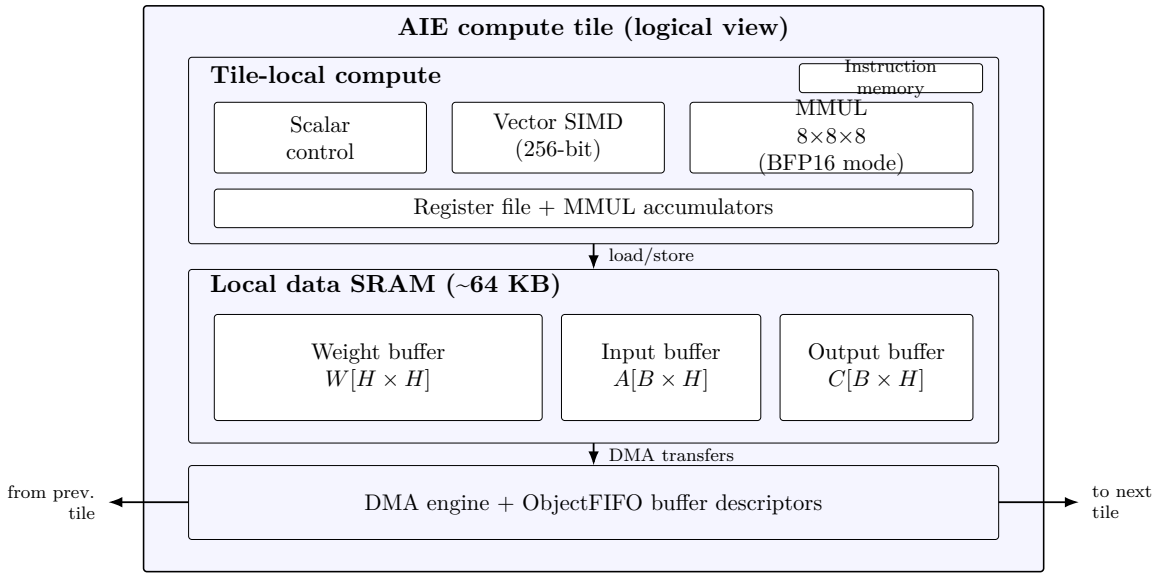

Figure 2: Logical view of one XDNA 2 compute tile: local SRAM, MMUL-backed compute, and explicit DMA / FIFO data movement.

Constraint	Why it matters	Choice in this project
~64 KB per tile	Weight matrix, activations, and scratch space must fit locally	Choose $H = 160$: 50 KB weights + 5 KB activations + headroom
8×8 MMUL-friendly layout	Host tensors must be packed in blocked layout; dimensions should be multiples of 8	Choose $B = 8$ and $H = 160$; use `to_tiled()` / `from_tiled()`
Explicit routing	Every inter-tile handoff has to be representable as a FIFO / route	Use a single snake of plain `ObjectFifo`s across all 32 tiles
Invocation overhead	Small calls waste hardware; fixed overhead dominates when the batch is tiny	Accept this for clarity; frame `B=8` as pedagogical, not peak-throughput
Current runtime semantics	Weights are written once per NPU call, not kept resident across many calls	Weight-stationary during a call; persistent weights remain future work

Table 3: This is the minimal constraint set needed to understand the rest of the document. Anything else belongs in an appendix, the code, or the logbook.

```
def forward(self, x):
    x = x.view(x.size(0), -1)
    x = self.embed(x)
    for layer in self.layers:
        x = layer(x)
    return self.head(x)
```

That symmetry matters. The main reason this whitepaper is easy to write at all is that the conceptual model and the deployed model are so close to one another.

3.2 Snake routing through all 32 tiles

The compute tiles are not used as eight independent columns in the current main branch. Instead, they are chained into one long serpentine path. The activation enters the first tile, flows through all 32 layers, and leaves the last tile. This is the cleanest way to make "one weight matrix per tile" literally true.

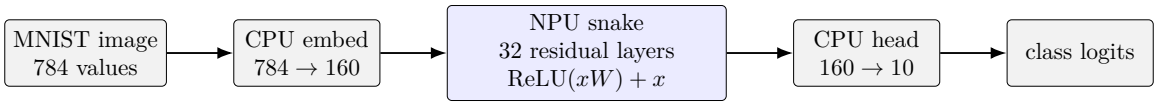

Figure 3: Host/NPU split in the current implementation: CPU embedding, 32 NPU residual layers, then a CPU classification head.

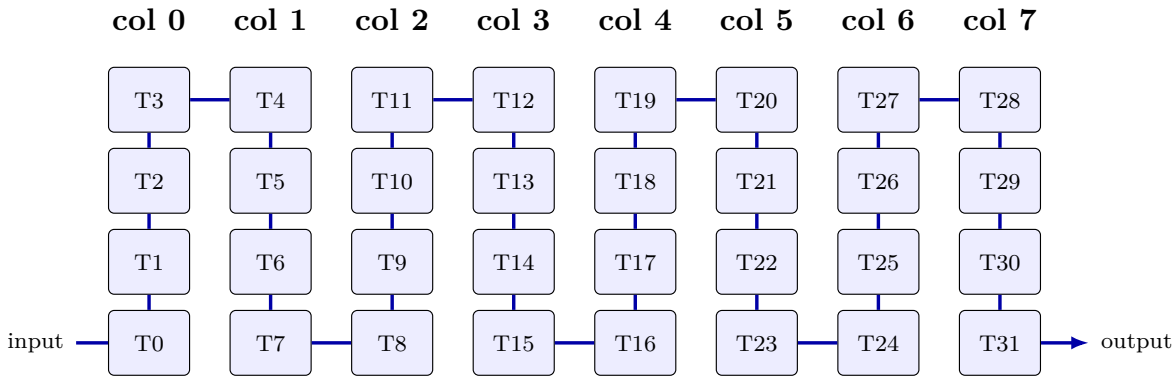

Figure 4: The 32-tile snake used in `resmlp/design.py`. Even-numbered columns are traversed bottom-to-top; odd-numbered columns top-to-bottom. Each node is one residual layer with its own weight matrix.

In code, this ordering is generated by a tiny helper:

```
def snake_tile_order(num_cols):
    tiles = []
    for col in range(num_cols):
        rows = range(2, 6) if col % 2 == 0 else range(5, 1, -1)
        for row in rows:
            tiles.append((col, row))
    return tiles
```

That function is deceptively simple. It is the point where the network architecture stops being abstract and starts matching the physical device.

3.3 Why $H = 160$ and $B = 8$

The hidden dimension $H = 160$ is chosen by SRAM, not by theory. A single tile must hold one 160×160 bf16 weight matrix plus the input and output activation buffers for a batch of 8 samples. Figure 5 shows the budget.

Numerically:

- weight matrix: $160 \times 160 \times 2 = 51{,}200$ bytes ≈ 50.0 KB,
- input activations: $8 \times 160 \times 2 = 2{,}560$ bytes,
- output activations: $8 \times 160 \times 2 = 2{,}560$ bytes.

This lands at roughly 55 KB before stack and miscellaneous overhead, leaving comfortable headroom under the tile's local memory limit.

The batch size $B = 8$ follows from the MMUL granularity. The current kernel uses $8{\times}8{\times}8$ blocking and expects dimensions divisible by 8. Choosing $B = 8$ is therefore the smallest clean batch that does not waste the MMUL shape. It is emphatically not the throughput-maximizing choice. It is the smallest choice that keeps the mapping simple and the tutorial honest.

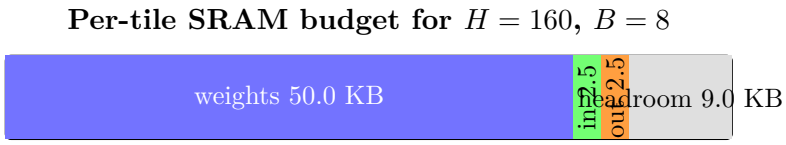

Figure 5: The key design choice in one picture. The weight matrix dominates the budget; activations are small enough that one tile can hold one full layer comfortably.

3.4 BFP16 and tiled host layouts

The main branch compiles the kernel with `-DAIE_API_EMULATE_BFLOAT16_MMUL_WITH_BFP16`. In practice this means that host-side weights and activations are arranged into blocked 8×8 layouts, and the MMUL path uses a BFP16-style execution mode that aligns well with that blocking. The helper functions `to_tiled()` and `from_tiled()` in `resmlp/__init__.py` perform the required data reordering.

This choice trades a small amount of numerical fidelity for a much cleaner and more uniform hardware mapping. In the test harness, the difference between the NPU path and a float32 reference compounds across 32 layers into noticeable per-element mismatch, but it does not meaningfully degrade classification accuracy.

4 From network to NPU program

4.1 What one tile computes

The tile-local kernel in `aie_kernels/matmul_relu_skip.cc` implements the whole layer in two steps: a blocked matrix multiply with ReLU, followed by a vectorized residual add.

```
extern "C" void matmul_relu_skip_bf16(bfloat16 *a, bfloat16 *w, bfloat16 *c)
{
    static_assert(DIM_M % 8 == 0, "Batch␣must␣be␣multiple␣of␣8");
    static_assert(DIM_K % 8 == 0, "Input␣dim␣must␣be␣multiple␣of␣8");
    static_assert(DIM_N % 8 == 0, "Output␣dim␣must␣be␣multiple␣of␣8");

    ::aie::set_rounding(aie::rounding_mode::floor);
    matmul_relu<bfloat16, (DIM_M / 8), (DIM_K / 8), (DIM_N / 8)>(a, w, c);
    residual_add(a, c);
}
```

Two details matter here.

1. The current kernel is deliberately simple. It uses a 1×1 output blocking pattern rather than the more aggressive 2×2 expansion used in older throughput-oriented experiments.
2. The residual connection is explicit in the kernel. This keeps the NPU path numerically close to the PyTorch model instead of forcing the host to perform an extra add between layers.

4.2 How IRON describes the snake

The IRON design in `resmlp/design.py` mirrors the conceptual picture almost literally:

- one activation FIFO into the first tile,
- one activation FIFO out of the last tile,
- one activation FIFO between every pair of consecutive snake stages,
- one weight FIFO per column, split in the MemTile into four tile-local weight streams,
- one worker per compute tile.

The worker body itself is tiny:

Buffer	Meaning	Size per batch
Input activation	one $B \times H$ hidden-state batch entering tile 0	2.5 KB
Output activation	one $B \times H$ batch leaving tile 31	2.5 KB
Weight buffer	all 32 tiled $H \times H$ matrices packed together	1.56 MB

Table 4: What the current runtime moves for each batch. The current implementation is weight-stationary during a call, but not across calls: the full weight buffer is written again for every batch.

```
def worker_fn(of_in, of_out, of_w, kern):
    x = of_in.acquire(1)
    y = of_out.acquire(1)
    w = of_w.acquire(1)
    kern(x, w, y)
    of_w.release(1)
    of_in.release(1)
    of_out.release(1)
```

This is one of the nicest properties of the current implementation. There is no large orchestration layer hidden elsewhere. The dataflow graph is already the program.

An important subtlety is that the code uses plain `ObjectFifo`s for the activation chain, even across column boundaries. The compiler handles the required inter-column routing through the memory infrastructure; the Python code does not manually spell out every routing hop.

4.3 What moves during one inference call

The end-to-end inference path in `resmlp/infer.py` is:

1. load a trained PyTorch checkpoint,
2. run the CPU embedding layer,
3. convert the hidden activations to tiled bf16 layout,
4. write input activations and packed weights to the NPU buffers,
5. execute the NPU runlist,
6. read back the final hidden state and apply the CPU classification head.

The sizes are instructive.

This weight rewrite cost per batch explains the modest measured performance. The model is structurally "NPU-native": each layer maps directly to one physical tile. But the current runtime still pays host-side setup and buffer-write costs every batch. This is not a bug; it is simply where the current implementation stops.

5 Results and what they mean

5.1 Accuracy and speed

The accuracy result is the easy part: the hardware path matches the software model closely enough that the NPU version classifies MNIST just as well as the CPU baseline. That gives us confidence that the mapping is correct.

The speed result needs more interpretation. A batch of 8 through 32 hidden layers performs roughly

$$32 \times (2BH^2 + BH) \approx 13.15 \text{ million operations}$$

for $B = 8$ and $H = 160$, counting both the matrix multiply and the residual add. That is not much work for a device with a 25-TFLOP bf16 headline. The design is therefore dominated by

Metric	Value
Model parameters	946K
CPU test accuracy	97.1%
NPU test accuracy	97.16%
NPU stage latency in `infer.py`	0.33 ms per batch of 8
NPU stage throughput in `infer.py`	24,114 images/s
Device-side benchmark in `test.py`	55 GFLOPS for the full 32-tile snake

Table 5: The current main-branch results. The two performance numbers answer slightly different questions: `infer.py` includes host-side buffer writes for each batch, while the device-side benchmark isolates the NPU run itself more directly.

overheads: invocation cost, buffer writes, and the simple fact that $B = 8$ was chosen for clarity, not for saturation.

5.2 Why the current design is far from peak throughput

The right way to read the current performance is not "the NPU is slow." The right way to read it is "this particular example is optimized for transparency." Three factors dominate.

1. **Small batch.** $B = 8$ is the smallest neat MMUL-aligned batch, but it leaves little work to amortize fixed costs.
2. **Per-batch weight writes.** The current runtime writes the entire 1.56 MB weight buffer every batch, even though the weights do not change.
3. **Simple kernel scheduling.** The main branch uses the simplest clean kernel and routing strategy, not the most aggressively throughput-tuned one.

This is why the paper should not be read as an argument that "the best use of XDNA 2 is MNIST with $B = 8$." It is instead an argument that a clean end-to-end example can make the co-design problem legible.

5.3 Why the example is still valuable

Despite the modest utilization, the current design teaches several things very cleanly.

- A neural network can map directly to the physical tile graph with very little abstraction gap.
- The most important design question on this hardware is often SRAM budget, not parameter count in the abstract.
- A simple CPU/NPU split — CPU for irregular edges, NPU for the repetitive core — is often the cleanest way to proceed.
- Explicit data movement clarifies the true cost model: what stays local, what streams, and what the host still has to do.

That is precisely why this repository pivoted away from more fragmented earlier iterations and toward the current `resmlp/` example.

6 Conclusion and next directions

The cleanest story in this repository is not "we reached peak TOPS." It is "we built a neural network whose structure is dictated by the hardware, and we can explain every part of that mapping." That is already a useful result.

For future work, the next steps are straightforward:

- keep weights resident across calls instead of rewriting them every batch,
- increase batch size to better amortize invocation overhead,
- explore int8 variants,

- move to a task that is more representative than MNIST once the mapping is fully understood.

The older experimental history remains valuable — especially the throughput benchmarks and language-model work — but it belongs in `logbook.md` and the `experimental` branch. The main whitepaper should stay sharp: one device, one mapping, one working example, fully explained.

A Current full-NPU training implementation

The main text explains the cleaner mapping that the current whitepaper is built around: CPU embedding, a 32-tile residual snake on the NPU, then a CPU classification head. The code in this repository now also contains a more ambitious training path in `resmlp/train_npu.py`. That path moves the entire MNIST model onto the NPU for one training call: embedding, residual stack, classification head, loss, backward pass, and SGD updates.

This appendix documents that implementation as it exists today. The important point is not that every systems problem is solved. It is that the training computation itself now lives on the array. The remaining rough edges are mostly runtime-state and persistence problems, not missing backpropagation math.

A.1 Tile layout and layer mapping

The full training graph still uses all 32 compute tiles, but not in the same role assignment as the inference snake from the main text. In `resmlp/training_full_design.py`, the 32 tiles are partitioned as shown in Table 6.

Tiles	Role	What runs there
tile 0	embedding	`embed_forward_bf16` and `embed_backward_bf16`
tiles 1–30	hidden stack	one residual layer per tile: `matmul_relu_skip_bf16` forward and `residual_backward_and_update_bf16` backward/update
tile 31	head + loss	`head_forward_loss_bf16` and `head_backward_bf16`

Table 6: The current full-NPU training decomposition. The full path uses one embed tile, thirty residual tiles, and one head tile.

This means that `full-npu` is a 30-residual-layer training design, not the older 32-residual-layer inference snake described earlier. The difference is structural, not cosmetic: the two end tiles are now used for the irregular edges of the network instead of being forced into the residual pattern.

A.2 What one training call does on the array

One invocation of the full training program performs the whole forward and backward path on device.

1. The host streams a raw 8×784 MNIST batch into the embed tile.
2. The embed tile applies the $784 \to 32$ projection.
3. The activation flows through 30 residual tiles in snake order.
4. The head tile computes logits, softmax-cross-entropy quantities, class predictions, and the tiled `d_logits` buffer.
5. The backward pass starts at the head tile and propagates gradients back through the residual chain to the embed tile.
6. Each stage applies its own SGD update in place.

The backward kernels are the main novelty. The head kernel in `aie_kernels/head_backward.cc` computes

$$dY_{\text{hidden}} = d\text{logits}\, W_{\text{head}}^T$$

and then applies

$$W_{\text{head}} \leftarrow W_{\text{head}} - \eta\, Y_{\text{hidden}}^T d\text{logits}.$$

The residual kernel in `aie_kernels/residual_backward.cc` performs both gradient propagation through the skip-ReLU layer and the outer-product SGD update of the local 32×32 weight matrix. The embed kernel in `aie_kernels/embed_backward.cc` does the same for the 784×32 projection matrix.

In other words, the appendix-level claim is strong but precise: SGD is not simulated on the host after an NPU forward pass. The weight updates themselves are executed inside the AIE kernels.

A.3 How IRON wires the training graph

The IRON graph in `resmlp/training_full_design.py` is the clearest place to see how the training step is organized.

- `embed_wt` and `head_wt` are direct weight FIFOs.
- Residual weights enter as eight per-column FIFOs and are split in the MemTiles into 3 or 4 tile-local weight streams, matching the snake occupancy of each column.
- Activations form one forward chain of `ObjectFifo`s.
- Gradients form one backward chain of `ObjectFifo`s.
- Each residual tile stores a compact checkpoint buffer containing its input activation and ReLU mask, so the backward phase can reconstruct the local gradient update.
- A small `done_fifo` marks backward completion, and `head_wt_out` drains the updated head weights back to host-visible memory.

One subtle but important detail is the embed backward phase. The embed tile does not keep the full raw 8×784 batch as a forward checkpoint in local SRAM. Instead, the host re-streams `x_raw` during the backward phase. This is a good example of the actual programming style on the array: whenever a buffer is too awkward to keep resident, the design should make that data movement explicit.

A.4 What works today, and what does not

The current implementation is good enough to support the central claim of this appendix: the full network, the backward pass, and the SGD update rules can all run on the NPU in one training call. It is not yet a fully resident training loop across many batches.

The stable path in `run_full_npu_batch()` still uses a fresh `AIEContext` for every batch. That workaround remains necessary because reusing the same hardware context corrupts later invocations. The important refinement is that the wrapper no longer needs a full `DefaultNPURuntime.cleanup(` on every call. Evicting only the cached hardware contexts is enough to restore correct execution while keeping the instruction-buffer cache hot.

In the current stable configuration, host-visible persistence works reliably for `head_wt` only. The embed and residual weights are updated in tile memory during a call, but the system does not yet drain them back in a form that can be reused safely across calls. Earlier attempts either exceeded SRAM, failed routing, or exhausted DMA descriptor resources.

This changes the performance picture substantially. The NPU work itself still takes about 5 ms for the $B = 8$, $H = 32$ training design, but the wrapper around it is now only a small extra cost instead of a 250 ms teardown penalty. A limited 500-batch probe now completes in about 3.7 s while preserving the same learning behavior as the older fresh-runtime wrapper. So

the present result should be read as both a correctness milestone and evidence that the runtime overhead is no longer the dominant bottleneck for this path.

Even with that limitation, the path is no longer a dummy stub. In the current repository, the stable full-NPU training mode learns on MNIST instead of remaining at chance. A limited probe with 500 training batches and 250 evaluation batches reaches about 40% test accuracy, which is enough to show that the on-device training path is functionally real even if full weight persistence is not yet the final design.

B Compact glossary

Term	Meaning in this project
AIE tile	One compute tile of the XDNA 2 array: local memory + local compute + DMA connectivity
MemTile	The memory tile that sits between shim tiles and compute tiles; useful for weight distribution and routing
Shim tile	The interface tile that connects the array to host-visible memory and runtime infrastructure
ObjectFIFO	IRON abstraction for explicit data channels between producers and consumers
MMUL	Tile-local matrix-multiply datapath used by the AIE kernel intrinsics
BFP16	Execution mode used here to make 8×8×8 blocking clean and efficient
XCLBIN	Compiled device artifact loaded by XRT
XRT	Runtime that launches the compiled design on the NPU

Table 7: Only the terms that are repeatedly needed in the main text are kept here.

C Toolchain in one page

The software pipeline in this repository is short enough to summarize directly:

```
aie_kernels/matmul_relu_skip.cc
    -> LLVM-AIE object
    -> kernel archive

resmlp/design.py
    -> Python-generated MLIR-AIE module

MLIR-AIE + aiecc
    -> .xclbin + instruction binary

XRT runtime
    -> buffer setup + runlist execution on the NPU
```

The IRON project and the MLIR-AIE ecosystem are the main external dependencies for the hardware side of this repository [1, 2]. The Linux AMD NPU documentation is also useful for understanding the runtime and driver-facing view of the device [3].

References

[1] AMD, *IRON repository.*
https://github.com/amd/IRON

[2] Xilinx / AMD, *MLIR-AIE programming guide.*
https://github.com/Xilinx/mlir-aie/tree/main/programming_guide

[3] Linux kernel documentation, *AMD NPU / amdxdna*.
https://docs.kernel.org/next/accel/amdxdna/amdnpu.html

[4] AMD, *AI Engine documentation (programming model and architecture)*.
https://docs.amd.com/r/en-US/ug1273-versal-acap-design/AI-Engine

[5] *AMD XDNA overview*.
https://en.wikipedia.org/wiki/AMD_XDNA